Computational Thinking

Alexander Abuabara

Pedagogy

This session introduces Python as a notation for computational problem solving. Students should leave with a basic mental model of how a computer evaluates expressions, stores values, executes code, and turns a small problem specification into a repeatable program.

This session reinforce the idea:

Python is the notation; computation is the subject.

For each new piece of Python, ask:

1. What problem are we representing?
2. What values does the computer need?
3. What operation should it perform?
4. How can we tell whether the result makes sense?

Therefore, this session establishes a clean conceptual foundation for later work on control flow. It does not introduce conditionals, loops, functions, or collections unless time permits.

Learning Objectives

By the end of the session, students should be able to:

1. Explain, at a basic level, what it means for a computer to execute a program.
2. Distinguish among Python code, an expression, a value, and a variable.
3. Run Python interactively in a notebook or IDE.
4. Predict and evaluate simple arithmetic expressions.
5. Identify basic numerical types using `type()`.
6. Assign values to variables and use variables in subsequent computations.
7. Write, save, modify, and rerun a short Python program.
8. Apply the computational-thinking cycle:

   problem → **representation** → **instructions** → **execute** → **inspect** → **revise**

Preparation

- Access Python 3 in Jupyter, VS Code, Spyder, Positron, or another environment.
- Check the Jupyter notebook: `1_computational_thinking.ipynb`.
- If using notebooks, check how to run a cell.
- If using an IDE, check how to run both a single expression interactively.

Agenda

1. What does it mean to compute?
2. Python as an executable language; notebook/IDE/script demonstration.
3. Predict, then run arithmetic expressions.
4. Objects, values, and numerical types.
5. Variables, assignment, and notebook state.
6. Build a tiny computational solution.
7. Debugging exercise.
8. Wrap-up and exit ticket.

1 What Does It Mean to Compute?

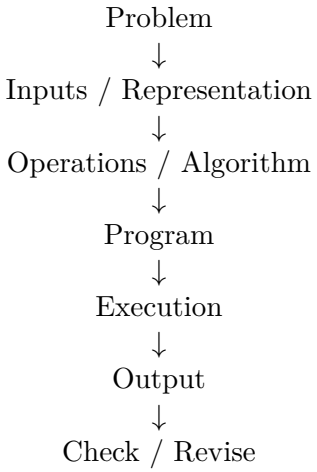
> A dataset contains 1,250 records. Processing one record takes approximately 0.018 seconds. Approximately how long will the entire dataset take to process?

One minute to solve the problem manually.

- What information did you use?
- What operation did you perform?
- What sequence of steps would you give a computer?
- What would change if the dataset contained 2.5 million records?

The instructional purpose is to distinguish solving a single instance manually from specifying a process that can be executed repeatedly.

Conceptual Model



Computational thinking means describing problems and solutions precisely enough that some or all of the solution can be carried out computationally.

2 Python as an Executable Language

Open the class notebook or Python console and run:

```
2 + 3
```

Then:

```
10 * 4
```

Ask: *What did Python do?*
Introduce three working terms:

Expression
Code that evaluates to a value.

Value
The result produced by evaluating an expression.

Interpreter
Software that reads and executes/evaluates Python code.

Notebook Versus Script

Notebook example:

```
distance = 150
time = 3
distance / time
```

Saved script example:

```
distance = 150
time = 3
speed = distance / time

print(speed)
```

Ask:

> Why do we typically need `print()` in a script, while the final expression in a notebook cell may display automatically?

Emphasize the conceptual distinction:

Interactive environment	**Script**
write → execute → inspect → modify → execute	write a sequence of instructions → execute the program

3 Predict, Then Run

Students open the provided notebook or an interactive Python console.

Rule for the activity:

> Do not run an expression until you have predicted its result.

This turns the interpreter into a tool for checking reasoning rather than trial-and-error guessing.

Arithmetic Expressions

Students predict and then execute:

```
3 + 4
3 * 4
3 + 4 * 2
(3 + 4) * 2
10 / 4
10 // 4
10 % 4
2 ** 5
```

Discuss:

- operator precedence,
- parentheses,
- `/` versus `//`,
- remainder with `%`,
- exponentiation with `**`.

Ask students to translate expressions into words. For example, `10 % 4` can be interpreted as finding the remainder after dividing 10 into groups of 4.

4 Objects, Values, and Types

Run:

```
type(3)
type(3.0)
type(10 / 2)
```

Ask:

> Are `3` and `3.0` the same thing computationally?

They may represent the same mathematical quantity, but Python stores and treats them as different numerical types.

Working Vocabulary

`3`	integer value (`int`)
`3.0`	floating-point value (`float`)
`2 + 3`	expression
`5`	value produced by the expression

Then ask students to predict both value and type:

```
2 + 3.0
type(2 + 3.0)
```

Quick Checkpoint

What will Python report?

```
type(7 / 2)
```

A. `int`

B. `float`

C. error

D. impossible to determine

Students vote before the code is executed.

5 Variables and Assignment

Introduce:

```
records = 1250
seconds_per_record = 0.018
```

Then:

```
total_seconds = records * seconds_per_record
total_seconds
```

Avoid initially describing `=` as mathematical equality. Instead:

> `records = 1250` associates the name `records` with the value `1250`.

Demonstrate reassignment:

```
records = 1250
records

records = 2500
records
```

Notebook State

Demonstrate:

```
x = 10
```

Then, in a separate cell:

```
x * 2
```

Edit the first cell to:

```
x = 100
```

but do not run it. Run the second cell again and discuss the result.
Key message:

> Computers execute what you run, not what you intended to run.

Use this moment to explain that notebook state depends on execution history, not merely visible cell order.

6 Build a Tiny Computational Solution

Lab Problem: Processing-Time Estimator

Scenario: A data-processing job has a known number of records and an estimated processing time per record. Write a short Python program that estimates the total processing time.

Step 1: Identify Inputs and Output

Students write:

```
Input:
    number of records
    seconds per record

Output:
    estimated processing time
```

Then:

```
records = 1250
seconds_per_record = 0.018
```

Step 2: Compute

```
total_seconds = records * seconds_per_record
```

Step 3: Display

```
print(total_seconds)
```

Step 4: Improve the Representation

```
total_minutes = total_seconds / 60
print(total_minutes)
```

Step 5: Test with Another Case

Change:

```
records = 1_000_000
```

Rerun the program.
Ask:

> Which parts of your program represent the particular problem instance, and which parts represent the general computational solution?

Desired distinction:

- **Data/parameters:** `records`, `seconds_per_record`
- **Computation:** `records * seconds_per_record`

Lab Challenge A: Predict Before Executing

Before running:

```
records = 100
seconds_per_record = 0.25

total_seconds = records * seconds_per_record
total_minutes = total_seconds / 60
```

Students record predictions for `total_seconds` and `total_minutes`, then verify them.

Lab Challenge B: Modify the Model

Add:

```
processors = 4
parallel_seconds = total_seconds / processors
```

Ask:

> What assumption is the program making?

Important answer: it assumes ideal scaling, so four processors provide a fourfold speedup.

Computational-thinking lesson: A program implements a model, and the model contains assumptions.

Lab Challenge C: Write a Small Script

Students choose one:

- Fahrenheit → Celsius
- miles → kilometers
- number of files × average file size → total storage
- records / elapsed seconds → records per second

Requirements:

- at least two variables,
- at least two expressions,
- at least one newly computed variable,
- at least one `print()` statement,
- meaningful variable names.

Avoid:

```
x = 100
y = 5
z = x / y
```

Prefer:

```
records_processed = 100
elapsed_seconds = 5
records_per_second = records_processed / elapsed_seconds

print(records_per_second)
```

7 Debugging Micro-Exercise 65–70 min

Give students these three examples.

Example A

```
records = 1000
seconds = 20
rate = records / Seconds
print(rate)
```

Example B

```
records = 1000
seconds = 20
rate = records seconds
print(rate)
```

Example C

```
records = 1000
seconds = 20
rate = seconds / records
print(rate)
```

Ask students to classify the issue:

- A: name/case error,
- B: syntax error,
- C: program runs, but the computation is logically wrong.

Emphasize that successful execution does not guarantee a correct program.
Establish the recurring course habit:

Predict → Run → Inspect → Explain

rather than:

Run → hope

8 Wrap-Up and Exit Ticket 70–75 min

Return to the opening problem and ask students to describe the solution without Python:

1. Represent the number of records.
2. Represent the processing time per record.

3. Multiply them.
4. Report the result.

Then map those ideas to Python:

```
records = 1250
seconds_per_record = 0.018

total_seconds = records * seconds_per_record

print(total_seconds)
```

Exit Ticket

1. Predict

```
x = 8
y = 3
z = x / y
```

What is the type of `z`?

2. Explain

In one sentence, explain the difference between:

```
3 + 4
```

and:

```
result = 3 + 4
```

3. Computational Thinking

For your lab program, identify:

```
Input(s):
Operation(s):
Output:
One assumption:
```

Suggested Instructor Notebook Structure

```
00 - Today's question
     "How do we turn a simple problem into something
      a computer can execute?"

01 - First Python expressions
02 - Predict -> run -> explain
03 - Numerical types
04 - Variables and assignment
05 - Notebook state
06 - Guided lab: processing-time estimator
```

```
07 - Lab challenges
08 - Debugging exercise
09 - Exit ticket
```

The notebook should remain activity-oriented rather than becoming a page of lecture notes. Each conceptual section should be followed quickly by executable work.

Core Vocabulary

Term	Working Definition
Program	A sequence of instructions that a computer can execute.
Expression	Python code that evaluates to a value.
Value	A piece of data produced or manipulated by a program.
Type	A category that determines how a value is represented and can be used.
Variable	A name associated with a value.
Assignment	Associating a name with a value.
Interpreter	Software that executes or evaluates Python instructions.
Script	Python code saved as a program and executed.
Notebook	An interactive document containing executable code cells and explanatory content.

Extra

Convert this word problem into a program:

> A sensor generates 240 observations per minute. Each observation requires 16 bytes. Estimate the storage required for one hour of observations.

Before coding, require:

```
Inputs:
Operations:
Output:
```

Then have students code, test, and exchange programs with another pair. The receiving pair changes one input and predicts the new output before running the program.

This provides a natural transition to the next class question:

> Once we can represent data and evaluate expressions, how can a program make decisions or repeat work?

References

[1] John V. Guttag. *Introduction to Computation and Programming Using Python: With Application to Computational Modeling and Understanding Data*. Third edition, MIT Press. https://mitpress.mit.edu/9780262542364/introduction-to-computation-and-programming-using-python/